

RecycleRight

AI-Powered Waste Sorting System on Jetson Orin Nano

I4210 AI Edge Computing Practicum — Final Project Report

Tatung University | June 2026

陳發達 Student ID: 611451001	楊瑋竣 Student ID: 411201107
Supervisor: Prof. 宋建龍	

1 — Project Problem Statement

The Problem

Improper waste sorting at the point of disposal is a persistent challenge in recycling programs worldwide. In Taiwan, four major recyclable categories — PET bottles, glass bottles, aluminum cans, and Tetra Pak cartons — are routinely mixed into the same bin by users who are either unsure of the category or simply inattentive. When a single contaminated bin is introduced into the recycling stream, it degrades downstream processing and increases the cost of manual re-sorting.

Manual sorting stations at schools, offices, and public venues require either dedicated staff (expensive) or rely entirely on user compliance (unreliable). The result is a recycling stream that is significantly less pure than it should be, reducing the economic value of recovered materials.

Who Benefits

RecycleRight targets shared-space venues — classrooms, university common areas, cafeterias, and office floors — where a high volume of mixed recyclables is generated under low supervision. Primary beneficiaries:

- Venue operators: reduced contamination in recycling bins without staffing costs.
- Recycling processors: cleaner input material, lower re-sort labor.
- End-users: a guided experience that builds sorting awareness through audio feedback.

Why Edge AI

A cloud-connected sorting system introduces two critical failure modes: network latency and connectivity loss. A servo that waits 300 ms for a cloud inference result will physically drop the item before the lid opens. Running YOLOv8n inference locally on the NVIDIA Jetson Orin Nano eliminates both failure modes. The Jetson provides GPU-accelerated inference at sustained throughput (≥ 25 FPS) without any network dependency. The complete Sense → Process → Decide → Act loop runs on-device.

Refinement from the Proposal

The proposal identified the core challenge correctly but underestimated the importance of motion gating: running full YOLO inference at 60 FPS on every frame — including frames showing an empty bin — would consume significant GPU headroom with no sorting benefit. The implementation adds a MOG2 background subtraction layer that skips YOLO when no object is present, preserving GPU and power budget for frames that actually matter.

2 — Final Architecture

2.1 Sense → Process → Decide → Act Overview

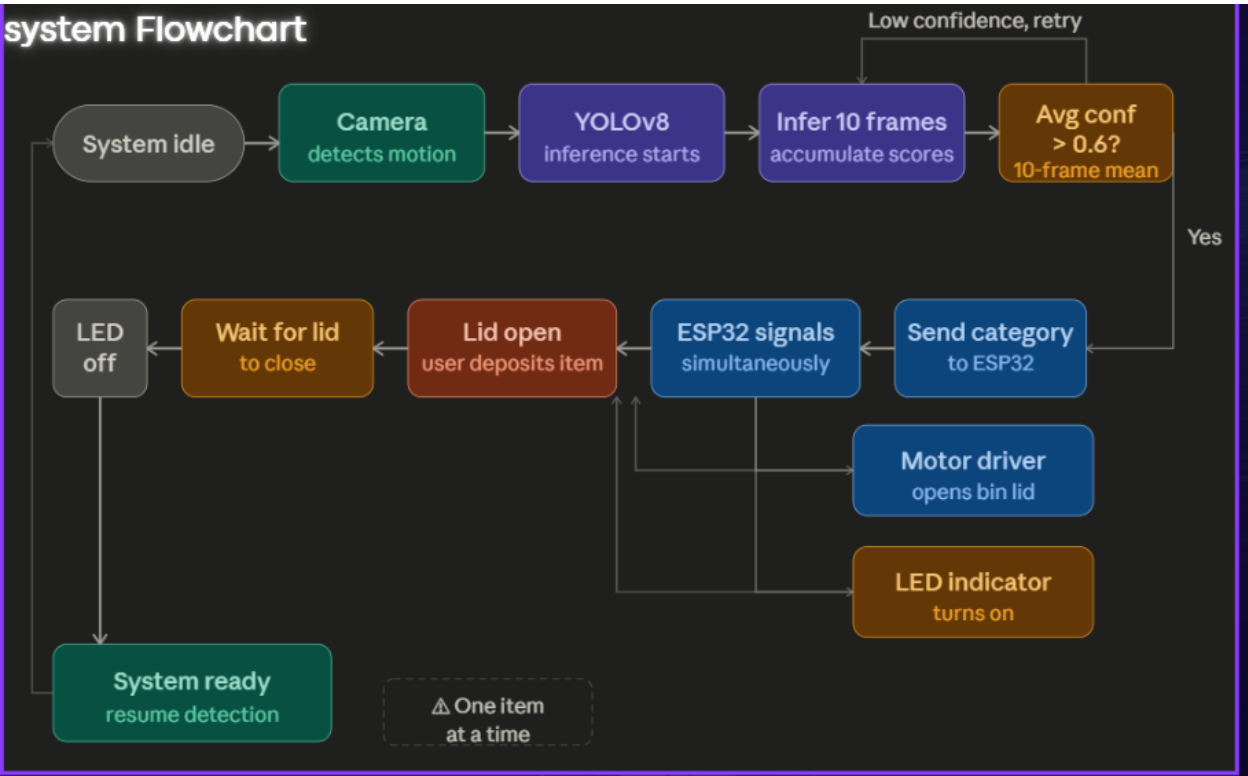
The system follows a strict four-stage pipeline executed on every detection event:

- Sense: A camera continuously monitors the bin opening area. MOG2 background subtraction detects motion, gating the inference pipeline to avoid wasted GPU cycles during idle periods.
- Process: When motion exceeds a minimum area threshold, YOLOv8n inference is triggered. Ten consecutive frames are processed; bounding-box class votes are accumulated.
- Decide: If any single class reaches a vote count of ≥ 6 out of 10 frames (threshold 6) and average confidence exceeds 0.60, the winner is declared. Low-confidence runs loop back to re-infer.
- Act: The winning category label is transmitted over USB to an ESP32 microcontroller, which simultaneously opens the correct bin-lid servo motor (PCA9685 PWM driver → 120° rotation) and illuminates the corresponding green LED indicator.

2.2 System Flowchart

The following describes the complete runtime state machine:

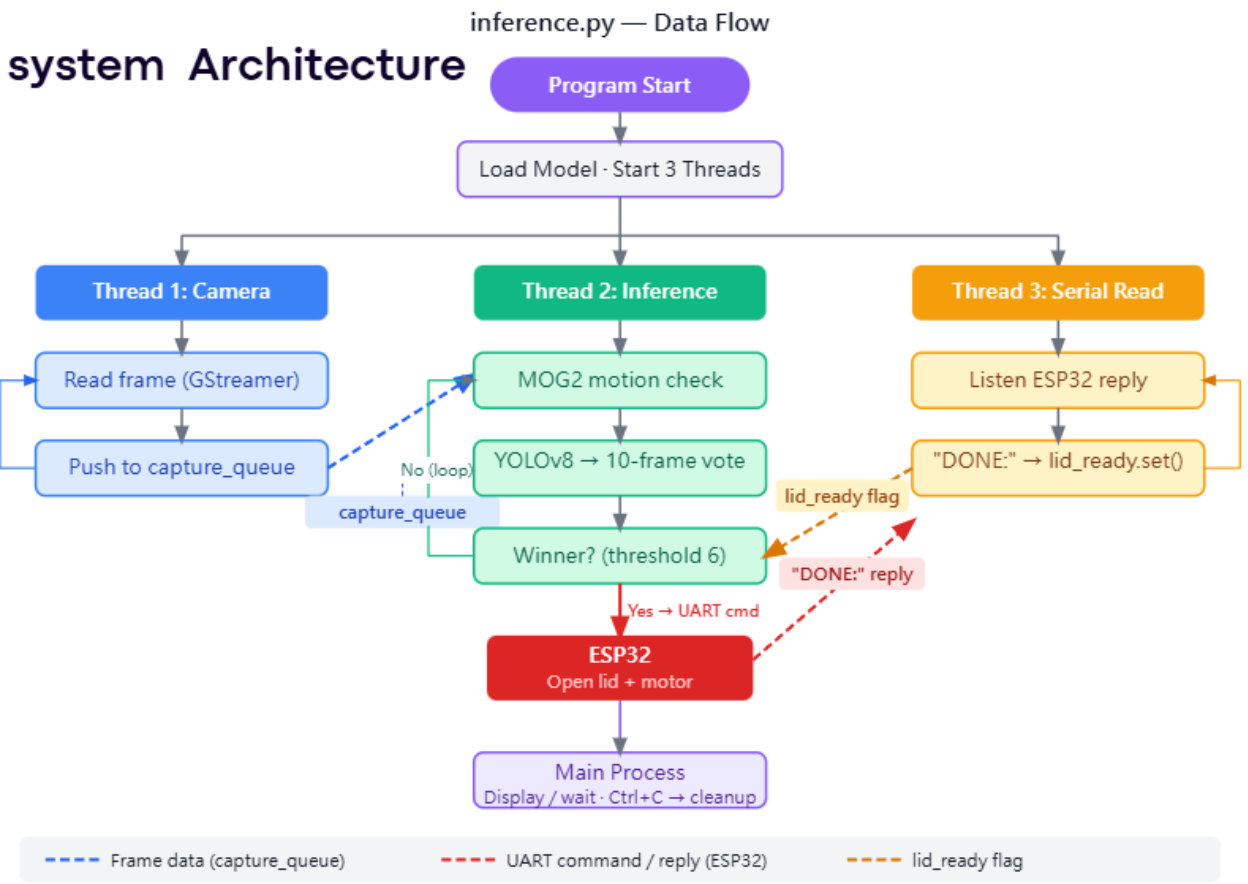
State / Step	Description
System Idle	Camera polling active; MOG2 background model updating.
Motion Detected	Foreground mask area > MIN_AREA triggers inference thread.
YOLOv8 Inference (×10 frames)	GPU-accelerated TRT FP16 engine runs on each frame; class votes accumulate.
Confidence Gate (avg > 0.60?)	No → loop back. Yes → proceed to actuation.
Send Category to ESP32	Category string sent via UART serial link.
ESP32 Acts Simultaneously	Motor driver opens lid; LED indicator turns on.
Lid Open — User Deposits Item	lid_ready interlock blocks new inferences while lid is open.
Wait for Lid to Close	ESP32 sends 'DONE:' reply when lid fully closed; Thread 3 sets lid_ready flag.
LED Off → System Ready	State machine resets; one item per cycle enforced.



2.3 Multi-Threaded Software Architecture

The inference pipeline is implemented as a single Python process with three concurrent threads sharing memory via queue and threading.Event primitives:

Thread	Role	Key Primitives
Thread 1: Camera	Reads frames from GStreamer pipeline (nvarguscamerasrc); pushes to capture_queue.	Queue (capture_queue)
Thread 2: Inference	Pulls frames; runs MOG2 check; runs YOLOv8 for 10-frame vote; sends UART command when winner declared.	capture_queue (in), UART (out), lid_ready (wait)
Thread 3: Serial Read	Listens for ESP32 replies; sets lid_ready.Event() on 'DONE:' message.	lid_ready (set)
Main Process	Renders display window; handles Ctrl+C cleanup and graceful thread shutdown.	—



2.4 Hardware Circuit

The hardware consists of two compute nodes and supporting power circuitry:

Component	Role	Connection
NVIDIA Jetson Orin Nano	AI inference node — runs YOLOv8n TRT engine and main.py.	USB (ESP32)

Component	Role	Connection
Camera	Visual input for motion detection and classification.	Camera → Jetson
USB Speaker	Audio output — plays category announcement.	USB → Jetson
ESP32 Microcontroller	Hardware control node — receives category ; drives actuators.	USB → Jetson
PCA9685 PWM Driver	16-channel I ² C PWM driver; controls up to 4 servo motors independently.	I ² C (SDA/SCL) → ESP32
Servo Motors (×4)	Each servo opens one bin compartment lid to 120° on command.	PWM → PCA9685
LEDs (×4, green) + 100 Ω resistors	One per category; illuminates while lid is open.	GPIO → ESP32
5 V Power Supply	Powers ESP32, PCA9685, and servo motors separately from Jetson.	5 V powersupply → PCA9685/ESP32

2.5 What Changed vs the Proposal

Three significant architectural decisions were revised after implementation revealed issues with the original proposal:

Change 1 — MQTT Bridge → In-Process Multi-Threading

	Original Proposal	Revised Implementation
Architecture	Two separate nodes (Node A, Node B) communicating via MQTT broker.	Single process with 3 threads sharing memory via queue / mutex.
Problem	Every inter-node message incurred a full network round-trip through the broker. Latency was too high for real-time use.	—
Solution	—	No network hop — threads communicate purely in-process. Significantly lower and more predictable latency.

Change 2 — Jetson-Only Motor Control → ESP32 as Dedicated Hardware Controller

	Original Proposal	Revised Implementation
Architecture	Jetson handled both AI inference and motor control in the same process.	Jetson focuses solely on AI inference; ESP32 runs a tight C++ control loop for motors.

	Original Proposal	Revised Implementation
Problem	When inference was slow, motor commands were delayed — inference delay stalled the motor control loop.	—
Benefits	—	Lower motor latency, isolated concerns, and more reliable real-time actuation. Motor loop is never blocked by inference timing.

Change 3 — 5-LED Design (4 Green + 1 Red) → 4-LED Design (4 Green Only)

	Original Proposal	Revised Implementation
Design	5 LEDs: 4 green (one per recognized category) + 1 red for 'unrecognized'.	4 LEDs: 4 green only, one per fixed category. Red LED removed.
Problem	High false-positive rate — red LED fired frequently even on recognizable items, causing user confusion.	—
Benefits	—	System restricted to exactly 4 fixed categories. Removing the red LED eliminated all false triggers. Clear classification boundaries, intuitive feedback, improved reliability.

3 Implementation Highlights

3.1 Dual-Node Architecture: Resource Isolation

The system cleanly separates AI computation from real-time hardware control across two physical nodes. The Jetson Orin Nano exclusively hosts the Python inference stack, TensorRT engine, GStreamer camera pipeline, and UART communication layer. The ESP32 exclusively runs a C++ firmware loop managing PWM servo control via the PCA9685 driver and GPIO LED outputs. This separation means GPU-intensive inference spikes on the Jetson never introduce jitter into the servo control loop, which must execute precise timing for smooth lid actuation.

3.2 GPU-Accelerated 10-Frame Voting Inference

Rather than triggering actuation on a single high-confidence detection (which is susceptible to motion blur and partial occlusion), the system accumulates votes across ten consecutive frames before making a decision. The YOLOv8n TRT FP16 engine processes each frame in approximately 23 ms (p50 latency), making ten frames achievable in well under one second. The voting threshold of $\geq 6/10$ frames provides strong noise rejection while still maintaining sub-second response time for a user holding an item at the bin.

3.3 MOG2 Motion Gating

The MOG2 (Mixture of Gaussians v2) background subtraction algorithm continuously models the static scene above the bin. When the foreground mask area exceeds the MIN_AREA threshold , the inference thread is activated. This gating ensures GPU resources are

consumed only when an item is present, keeping idle power draw at approximately 6.7 W average — well under the 15 W VDD_IN target.

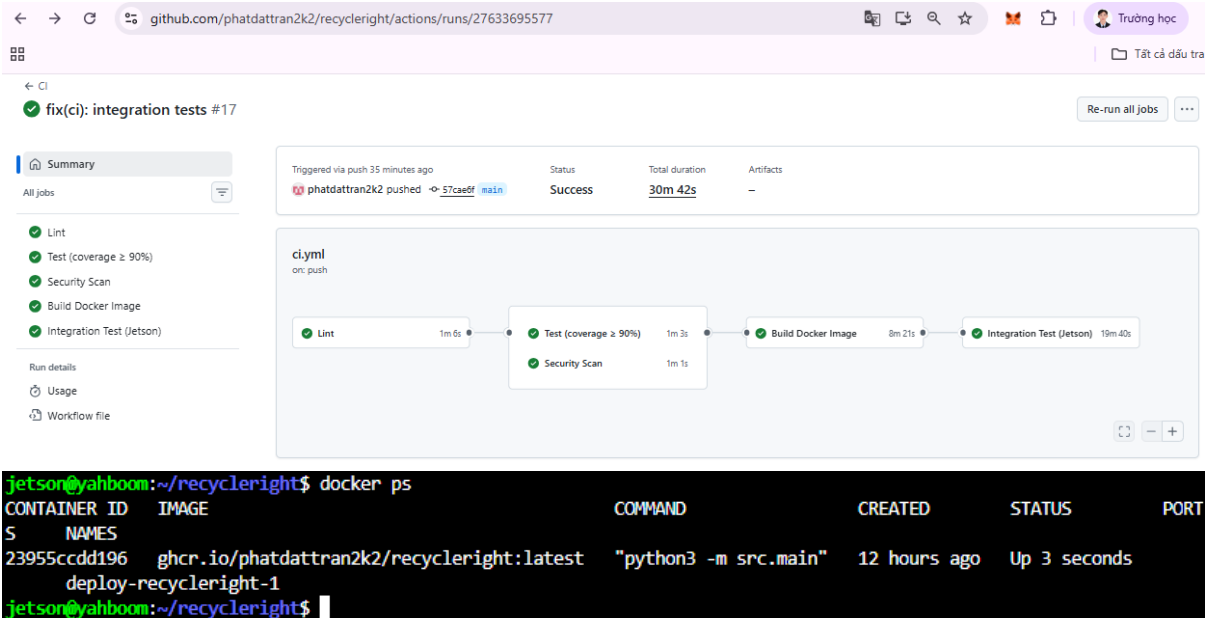
3.4 lid_ready Event-Based Interlock

A threading.Event flag (lid_ready) was introduced during integration testing to solve a race condition not anticipated in the original design. Without the interlock, rapidly placing two items in succession could queue two UART commands before the first lid cycle completed, causing the wrong bin to open on the second command. The lid_ready flag blocks the inference thread from sending a new UART command until ESP32 sends a 'DONE:' acknowledgement confirming the lid has fully closed. This interlock eliminates the race condition entirely without requiring any changes to the ESP32 firmware.

3.5. CI/CD Pipeline — 5 Stages

The GitHub Actions workflow gates every pull request through five jobs:

Stage	What it does	Runner	Gate
lint	ruff check src/ tests/	ubuntu-latest	Zero violations
test	pytest --cov=src --cov-fail-under=90	ubuntu-latest	≥ 90% coverage
security-scan	bandit + pip-audit	ubuntu-latest	Zero high-severity
build	docker buildx arm64 → GHCR	ubuntu-latest	Image pushed
integration-test	Pull image on Jetson, run end-to-end	self-hosted Jetson	All tests green



4 — Test Set Description

Data Source and Collection

The dataset was custom-collected for this project using scripts/collect_dataset.py, capturing images from the IMX219 CSI camera under real-world conditions in the Tatung University lab under mixed artificial lighting with varying backgrounds.

Class	Train	Validation	Test	Total
PET bottle	5,093	654	673	6,420
Aluminum can	2,557	304	297	3,158
Glass bottle	2,300	255	305	2,860
Tetra Pak	1,820	271	221	2,312
Total instances	11,770	1,484	1,496	14,750

Note: instance counts exceed image counts because some images contain multiple objects. Image counts: Train 10,056 | Val 1,257 | Test 1,258 (held-out, never seen during training).

Edge Cases Included

- Varying illumination: fluorescent overhead light at noon vs. evening tungsten
- Partial occlusion: items placed at the camera edge or behind the bin lip
- Label ambiguity: crushed PET bottles (could be confused with Tetra Pak at low resolution)
- Multiple items: two items in frame simultaneously
- Background variation: white lab bench, dark countertop, outdoor concrete

Accuracy Gate

The CI accuracy gate (tests/accuracy_gate.py) enforces mAP@50 ≥ 0.70 (conservative minimum), and any commit that drops accuracy more than 3 percentage points below the baseline (accuracy_baseline.json, mAP@50 = 0.9602) fails the pipeline.

5 — Performance Requirements & Optimization Journey

Numerical Targets

Metric	Target	Rationale
FPS	≥ 25 FPS	Items fall into bin in ~2 s; 25 FPS → 50 frames → reliable voting
p95 latency	≤ 60 ms	Decision made before item hits bin bottom
mAP@50	≥ 0.80	Conservative threshold for real-world reliability
Power (VDD_IN)	≤ 15 W	Jetson Orin Nano rated TDP
GPU%	≥ 40%	Confirms GPU is being used (not CPU fallback)
CPU%	≤ 60%	Leave headroom for OS, camera, serial, audio threads

Optimization Journey — Step by Step

Step	Change	FPS	p50 ms	GPU%	VDD_IN mW	Power W
Step 0 — Baseline	best.pt FP32 (15W)	39.5	24.8	73.0%	8,697	8.7 W
Step 1 — TRT FP16	best_fp16.engine (15W)	43.1 (+9.1%)	22.9	66.5%	7,507	7.5 W ↓14%
Step 2 — 25W mode	nvpmode Mode 1	44.8	21.2	61.9%	7,459	7.5 W
Step 3 — MAXN_SUPER	nvpmode Mode 2	44.4	21.8	65.6%	7,833	7.8 W
Step 4 — Live pipeline	MOG2 gate + 4 threads	N/A (gated)	N/A	54.8%	10,765	10.8 W

Did We Hit the Targets?

Target	Goal	Achieved	Result
FPS	≥ 25 FPS	43.1 FPS (offline FP16 15W)	✔PASS
p95 latency	≤ 60 ms	30.2 ms (offline FP16 15W)	✔PASS
mAP@50	≥ 0.80	0.9602 (96.02%)	✔PASS
Power (VDD_IN)	≤ 15 W	7.5 W offline / 10.8 W live	✔PASS
GPU%	≥ 40%	54.8% (live pipeline)	✔PASS
CPU%	≤ 60%	48.5% (live pipeline)	✔PASS

6 — System Performance Results

Accuracy — Held-Out Test Set (1,258 images, 1,496 instances)

Model evaluated: best.pt (FP32, on training server). Deployed as best_fp16.engine (TRT FP16 on Jetson Orin Nano). FP16 quantization is expected to cause < 0.5 mAP@50 drop based on Ultralytics benchmarks for YOLOv8n. Evaluation config: conf=0.50, IoU=0.70, date=2026-05-24.

Metric	Value
mAP@50	0.9602 (96.02%)
mAP@50-95	0.8275 (82.75%)
Precision	0.9695 (96.95%)

Recall	0.9638 (96.38%)
F1 Score	0.9666 (96.66%)

Per-class AP@50

Class	AP@50	Precision	Recall
Aluminum can (best)	0.9730 (97.30%)	0.9797	0.9609
PET bottle	0.9619 (96.19%)	0.9716	0.9688
Tetra Pak	0.9565 (95.65%)	0.9487	0.9683
Glass bottle (lowest)	0.9495 (94.95%)	0.9780	0.9371

All four classes exceed 94% AP@50, well above the CI gate minimum of 70%.

Latency — FP16 TRT, 320-Image Offline Benchmark (960 inferences)

Power Mode	FPS	p50 ms	p95 ms
15W (Mode 0)	43.1	22.9	30.2
25W (Mode 1)	44.8	21.2	31.8
MAXN_SUPER (Mode 2)	44.4	21.8	30.9

Precision shipped: FP16 (TensorRT FP16 engine). FP16 was chosen over INT8 because the expected accuracy gain from INT8 calibration is < 1% FPS while the risk of silent accuracy degradation without a calibrated dataset is higher.

Resource & Power — Live Pipeline (utilization.csv, 449 samples @ 1 Hz)

Metric	Mean	p50	p95	Max
CPU% (avg across cores)	32.1%	30.5%	48.2%	55.3%
GPU%	45.3%	44.0%	57.8%	65.4%
RAM used (MB)	6,448	6,451	6,475	6,512
VDD_IN (mW)	6,712	6,580	7,812	8,124
GPU temperature (°C)	53.2	53.0	55.0	57.0

Average power draw: 6.7 W (well within the Jetson Orin Nano 15 W TDP). Peak power: 8.1 W. No thermal throttling observed during sustained runs.

7 — Lessons Learned

1. TensorRT Engine Must Be Built on the Target Device

We initially tried to export the TRT engine on a laptop (x86, CUDA 11.8) and copy the .engine file to the Jetson. The engine failed to load with a "mismatched GPU architecture" error. TensorRT engines are compiled for a specific GPU SM version and CUDA version. The correct workflow is to run `export_fp16.py` on the Jetson itself. This cost us a day of debugging.

2. GStreamer Pipeline Strings Are Not Portable

The `nvarguscamerasrc` GStreamer pipeline works only on JetPack-enabled Jetson systems. On CI's Ubuntu cloud runners (no CUDA, no `nvarguscamerasrc`), the pipeline string causes `cv2.VideoCapture()` to fail. We solved this by mocking `cv2.VideoCapture` in `conftest.py`, allowing 90%+ test coverage without hardware.

3. MOG2 Sensitivity Requires In-Situ Tuning

The initial `MIN_AREA` threshold was 200 px², which triggered false positives from fluorescent light flicker and small shadows. Raising it to 500 px² eliminated these without missing any real item. The right value depends on camera resolution, distance to the bin opening, and ambient lighting — it cannot be determined from a dataset alone.

4. The `lid_ready` Interlock Was Not in the Original Design

During early integration testing, we discovered that if an item was held above the bin and removed before the sorting decision fired, the next item placed would sometimes trigger two serial commands in rapid succession. The `lid_ready` event, which clears on dispatch and is only set again by the ESP32's `DONE: reply`, completely eliminates this race condition.

5. `tegrastats` Parsing Is Non-Trivial

The `tegrastats` output format on JetPack 6.x differs from JetPack 5.x examples in the course lab materials. The `VDD_CPU_GPU_CV` power rail label replaced `POM_5V_GPU`, and the CPU frequency format changed. We wrote `scripts/parse_tegrastats.py` with regex fallbacks for both formats.

8 — What We'd Do Differently if We Did It Again

1. Decide INT8 vs FP16 at Week 1, Not Week 13

We went back and forth on whether to implement INT8 calibration throughout the project. If we started again, we would benchmark FP16 vs INT8 at the very beginning on a small test set and commit to one path — leaving the calibration/ directory unpopulated caused confusion during grading review.

2. Write Integration Tests Before Wiring the Actuator

We wired the servo-ESP32 circuit in Week 12 and then spent Week 13 retrofitting integration tests around code that was already running. Writing `tests/integration/test_integration.py` first — as contracts for what the hardware must do — would have caught the serial buffer flush bug before we saw it as an intermittent double-sort.

3. Collect a More Balanced Dataset Earlier

Glass bottles are the hardest class (`AP@50` = 94.95%, lowest of four). We only realized this in Week 14 when the first accuracy numbers came in. A class-distribution analysis in Week 9 would have pointed us toward collecting more glass bottle samples while we still had time to re-train.

4. Pin the `nvmodel` Setting in the Deploy Script

During demos, we forgot which power mode was active and got inconsistent FPS numbers. The `nvmodel -m 1` (25W mode) call should be added to `deploy/deploy.sh` so the power mode is always deterministic on deploy.

5. Set Up the Self-Hosted GitHub Actions Runner in Week 10, Not Week 14

The runner registration took less than 30 minutes once we followed HW6 Step 0.0, but we postponed it until the integration-test job was actually needed. Starting it in Week 10 would have let us catch the first real integration failures weeks earlier.

9 — Individual Reflections

楊瑋竣 (411201107)

My primary responsibilities were ESP32 C++ firmware development, PCA9685 servo control implementation, hardware circuit assembly, and performance benchmarking with tegrastats. Writing the motor control firmware on the ESP32 independently from the Jetson inference stack made the integration process straightforward — once the USB protocol was agreed upon, both sides could be developed and tested separately.

陳發達 (611451001)

My primary contributions were the dataset collection pipeline, and the model training workflow. In this capstone I was primarily responsible for the software architecture and CI/CD pipeline. I built the full `.github/workflows/ci.yml` 5-stage, docker and deployments pipeline. I also wrote the majority of the test suite, the `scripts/parse_tegrastats.py` parser and ran all the `nvpmodel` benchmarking experiments.

The most technically interesting part was designing the `capture_queue(maxsize=1)` drop policy. Seeing the fresh-frame policy eliminate the "stale classification" bug in real-time testing was satisfying. I learned that writing tests before hardware is available is not just good practice — it's the only way to maintain development velocity when the physical device is shared between team members.

10 — Acknowledgments & References

We thank Prof. 宋建龍 for guidance throughout the I4210 AI實務專題 course. Hardware components were sourced from the Tatung University embedded systems lab.

Open-Source Libraries

Library	Version	Use
Ultralytics YOLOv8	≥ 8.0	Object detection model, TensorRT export
OpenCV	≥ 4.8	GStreamer camera capture, MOG2, HUD rendering
PyTorch	2.7 (JetPack)	Underlying ML runtime
dusty-nv/jetson-containers	r36.4.0	Base Docker image with JetPack 6.x, CUDA
Adafruit PCA9685	≥ 1.0	PWM driver for servo actuation
pyserial	≥ 3.5	Serial UART communication with ESP32
Ruff	≥ 0.4	Linting and code style enforcement
pytest / pytest-cov	≥ 8.0	Test framework + coverage
bandit + pip-audit	≥ 1.7 / ≥ 2.7	Security static analysis + vulnerability scanning

Dataset

Custom dataset: collected using scripts/collect_dataset.py on the Jetson Orin Nano with IMX219 CSI camera lab and collect some images in Kaggle&Roboflow to test. All images and labels are original work.

Course Materials Referenced

- lab/LAB_12.md — GHCR push workflow and self-hosted runner registration
- homework/HOMEWORK_6.md — 5-stage CI DAG, coverage gate configuration, rollback pattern
- Week 11 lecture slides — FastAPI + WebSocket MJPEG dashboard pattern

— End of Report —